

고급 스케줄링

Advanced Scheduling — 파드를 "어디에" 둘지 내 마음대로

 Kubernetes in Action · Chapter 16

저자: Marko Lukša · 1판 기준 (2판/최신 버전 차이점 포함)

발표자: 박세우

CH 16 16장 미리보기 — 스케줄러를 "내 의도대로" 길들이기

기본 스케줄러는 알아서 잘 배치해줘요. 하지만 "이 노드는 비워둬", "GPU 노드에만 올려", "레플리카는 서로 다른 노드/존에 흩뿌려" 같은 내 의도를 넣고 싶을 때가 있죠. 16장은 그 도구 상자입니다.

🔴 **이 장을 왜 배우나:** 지금까지는 "파드를 몇 개 띄울지"에 집중했어요(레플리카). 이제는 "그 파드를 정확히 어느 노드에 둘지"를 다룹니다. 가용성(서로 다른 존에 분산), 성능(같은 노드에 모아 지연 줄이기), 비용/격리(전용 하드웨어 노드)를 직접 통제하게 돼요.

🍔 **쉽게 말하면:** 결혼식 좌석 배치도 짜기예요. 하객(파드)을 아무 테이블(노드)에나 앉혀도 되지만 — 신랑/신부 가족은 앞자리(전용 노드), 사이 안 좋은 사람은 멀리 떼어놓고(안티 어피니티), 친한 친구끼리는 같은 테이블에(어피니티) 앉히고 싶죠. 16장이 그 배치 규칙을 쓰는 법이에요.

이번 장에서 배우는 3가지 도구 + 신규 주제 (카드를 클릭하면 설명이 떠요)

16.1

① 테인트 & 톨러레이션 (밀어내기)

노드에 "출입 금지" 표식(taint)을 붙여 파드를 밀어냅니다. 그 표식을 건디는(toleration) 파드만 들어올 수 있어요.

이 장의 다른 도구(어피니티)는 전부 "파드가 노드를 고르는" 방향입니다. 테인트만 유일하게 노드가 파드를 거부하는 반대 방향이에요.

3요소

- **taint(노드에 부착):** `key=value:effect`
- **toleration(파드에 부착):** 그 taint를 건디는 표식
- **effect:** NoSchedule / PreferNoSchedule / NoExecute

핵심: toleration은 "갈 수 있는 자격"일 뿐 "가야 함"이 아닙니다. 전용 노드를 만들려면 `nodeAffinity`와 짝지어 써야 해요.

16.2

② 노드 어피니티 (끌어당기기)

파드가 "이런 노드가 좋아"라고 끌리는 규칙. `nodeSelector`의 강력한 상위호환이에요.

파드가 "이런 레이블을 가진 노드에 가고 싶다"를 표현합니다. `nodeSelector`가 단순 일치만 댔다면, 노드 어피니티는 연산자(In/NotIn/Exists/Gt/Lt)와 hard/preferred 두 단계를 지원해요.

- `requiredDuringSchedulingIgnoredDuringExecution` = 필수(hard)
- `preferredDuringSchedulingIgnoredDuringExecution` = 선호(soft, weight)

③ 파드 어피니티 / 안티 어피니티

"저 파드 **곁에** 두거나(affinity) **멀리** 떼어놓기(anti-affinity)". 노드가 아니라 **다른 파드**를 기준으로 배치해요.

"저 파드 곁에"(podAffinity) 또는 "저 파드 멀리"(podAntiAffinity)를 표현합니다. 같은 곳의 범위는 **topologyKey**로 정해요(노드/존/리전).

- **어피니티**: 캐시-웹을 같은 노드에 → 지연 감소
- **안티 어피니티**: 레플리카를 서로 다른 노드에 → 가용성

최신에선 "균등 분산"은 topologySpreadConstraints가 더 우수합니다.

④ 토폴로지 분산 제약 ★

안티 어피니티보다 똑똑하게 "존마다 균등하게" 흩뿌리는 최신 방식. 1판엔 없던

topologySpreadConstraints.

파드를 존/노드에 고르게 퍼 바르는 전용 도구.

maxSkew로 "도메인 간 개수 차이"를 제한합니다.

- **maxSkew**: 허용 최대 불균형
- **topologyKey**: 분산 기준(존/노드)
- **whenUnsatisfiable**: DoNotSchedule(hard) / ScheduleAnyway(soft)

안티 어피니티보다 유연하고, 큰 클러스터에서 성능 부담도 적습니다.

⑤ 스케줄링 프레임워크 ★

스케줄러 내부가 1판의 "predicate/priority"에서 **플러그인** 구조로 재설계됐어요. 프로파일도 여러 개 운용 가능.

1판의 predicate(필터)/priority(스코어)가 코드에 박혀 있던 것과 달리, 스케줄링 과정을 확장 지점(Filter, Score, PostFilter, Bind 등)으로 쪼개고 플러그인을 끼우는 구조로 재설계됐습니다.

우리가 쓴 taint·어피니티·분산 제약·선점도 전부 내장 플러그인입니다. 여러 **프로파일**을 한 스케줄러에서 운용할 수 있어요.

⑥ 우선순위 & 선점 ★

중요한 파드가 자리가 없으면 **덜 중요한 파드**를 쫓아내고 자리를 차지해요. **PriorityClass**.

파드에 중요도(정수)를 부여하고, 높은 우선순위 파드가 자리가 없으면 낮은 파드를 **축출(선점)**해 자리를 만듭니다.

- **PriorityClass** 리소스 → **priorityClassName**으로 파드에 부여
- **preemptionPolicy**: **Never** → 우선 배치만, 축출은 안 함
- 선점 로직은 프레임워크의 **PostFilter**에서 동작

🔴 **발표 흐름**: 먼저 스케줄러가 어떻게 일하는지 한 번 복습하고 → 1판의 3대 도구(테인트·노드 어피니티·파드 어피니티)를 차례로 → 마지막에 ★ 표시한 **2판/최신 신규 주제 4가지**를 묶어서 봅니다.

📖 이 슬라이드의 용어 풀이

노드 어피니티

파드가 "이런 레이블을 가진 노드에 가고 싶다"를 표현하는 규칙이에요. nodeSelector보다 표현력이 좋아 In/NotIn 등 연산자와 필수/선택 두 단계를 지원해요.

레이블

리소스(노드·파드 등)에 붙이는 key=value 꼬리표예요. 스케줄링 규칙(어피니티·셀렉터)이 이 레이블을 보고 대상을 고릅니다.

선점

높은 우선순위 파드가 배치될 자리가 없을 때, 스케줄러가 낮은 우선순위 파드를 축출해 자리를 만드는 것이예요(preemption).

스케줄러

"이 파드를 어느 노드에서 실행할까?"만 결정하는 컴포넌트예요. 파드를 직접 만들거나 실행하지는 않고, 조건에 맞는 노드를 골라 nodeName을 채워줘요.

스케줄링 프레임워크

스케줄러 내부를 여러 "확장 지점"(Filter, Score, PostFilter, Bind 등)으로 쪼개고, 각 지점에 플러그인을 끼워 동작을 바꾸는 최신 구조예요. 예전의 predicate/priority를 대체했어요.

안티 어피니티

"같은 종류의 파드가 이미 있는 곳은 피해라"는 규칙이에요. 레플리카를 서로 다른 노드/존에 흩뿌려 한 곳이 죽어도 살아남게 해요.

축출

실행 중인 파드를 노드에서 강제로 내보내는 것(eviction)이에요. NoExecute taint나 우선순위 선점에서 일어나요.

테인트

노드에 붙이는 "출입 통제 표식"이에요(taint). key=value:effect 형태로, 이 표식을 건디는 toleration이 없는 파드는 그 노드에 못 들어와요. 어피니티와 반대로 "밀어내는" 방향이에요.

토폴로지 분산 제약

파드를 존/노드에 고르게 퍼 바르는 최신 도구예요(topologySpreadConstraints). maxSkew로 "도메인 간 개수 차이"를 제한해, 안티 어피니티보다 유연하게 균등 분산해요.

톨러레이션

파드가 노드의 taint(꺼림칙한 딱지)를 "괜찮아, 참고 들어갈게"라고 받아주는 표식이에요(toleration). "견딘다"는 = "그 딱지를 참아낸다"는 뜻. taint와 짝이 맞아야 그 노드에 들어갈 "자격"이 생겨요. 단, 꼭 그 노드로 간다는 뜻은 아니예요.

파드 어피니티

노드가 아니라 "다른 파드"를 기준으로 배치하는 규칙이에요. "저 파드가 있는 곳에 같이 뒤"를 표현해요. (반대는 안티 어피니티)

effect

taint가 파드에 미치는 효과예요. NoSchedule(새 배치 금지) / PreferNoSchedule(되도록 금지) / NoExecute(실행 중인 것도 축출) 세 가지가 있어요.

IgnoredDuringExecution

"이미 실행 중이면 무시"라는 뜻이에요. 배치된 뒤 노드 레이블이 바뀌어 조건이 안 맞아도 파드를 쫓아내지 않아요. (실행 중 축출하는 RequiredDuringExecution은 아직 미구현)

maxSkew

토폴로지 분산 제약에서 "가장 많은 도메인과 가장 적은 도메인의 파드 개수 차이" 허용 최대치예요. 작을수록 더 뽁뽁하게 균등해져요.

nodeSelector

파드에 "이 레이블을 가진 노드에만 가라"고 적는 가장 단순한 방법이에요. 정확히 일치(AND)만 되고, 노드 어피니티가 그 상위호환이에요.

NoExecute

taint 효과 중 하나. 새 파드 배치를 막는 건 물론, "이미 실행 중인데 건디지 못하는 파드까지 쫓아내요(축출)". 노드 장애 시 자동 축출에 쓰여요.

NoSchedule

taint 효과 중 하나. 건디지 못하는 파드를 그 노드에 "새로 배치하지 않음"이에요. 이미 떠 있는 파드는 건드리지 않아요.

preemptionPolicy

우선순위 파드가 "남을 쫓아낼지" 정하는 정책이에요. PreemptLowerPriority(쫓아냄, 기본) / Never(우선 배치만 받고 선점은 안 함).

PreferNoSchedule

taint 효과 중 하나. "되도록 배치하지 말되, 다른 자리가 없으면 허용"하는 약한(soft) 버전이에요.

preferredDuringScheduling

"배치할 때 되도록 만족하면 좋은" 선호(soft) 규칙이에요. 못 맞춰도 배치는 되고, weight(1~100)로 선호 강도를 줘요.

PriorityClass

파드의 중요도(정수 값)를 정의하는 리소스예요. 파드는 `priorityClassName`으로 이걸 참조하고, 값이 클수록 스케줄링·자원 경쟁에서 우선해요.

requiredDuringScheduling

"배치할 때 반드시 만족해야 하는" 필수(hard) 규칙이에요. 맞는 노드가 하나도 없으면 파드는 Pending 상태로 멈춰요.

taint

영어로 "오염·얼룩"이라는 뜻이에요. 노드에 붙이는 "꺼림칙한 딱지(출입 통제 표식)"라고 보면 돼요. 이 딱지를 "괜찮아"라고 받아주는 `toleration`이 없는 파드는 그 노드에 배치되지 못해요.

toleration

파드가 노드의 `taint`를 "괜찮아, 참을 수 있어"라고 받아주는 표식이에요. "견딘다 = 그 노드의 꺼림칙한 조건을 참아낸다". `taint`와 `key/effect`가 맞으면 그 노드에 들어갈 자격이 생겨요.

topologyKey

파드 어피니티에서 "같은 곳"의 범위를 정하는 레이블 키예요. `hostname`이면 "같은 노드", `zone`이면 "같은 가용 영역"처럼 도메인 크기를 바꿔요.

weight

`preferred`(선호) 규칙의 가중치(1~100)예요. 여러 선호 조건의 `weight`가 합산되어 노드 점수가 되고, 점수가 높은 노드가 선택돼요.

whenUnsatisfiable

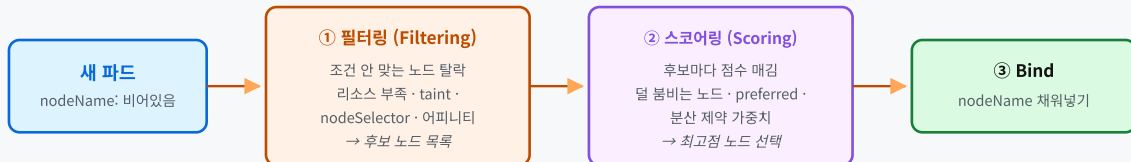
분산 제약을 못 맞출 때 어떻게 할지예요. `DoNotSchedule`(못 맞추면 Pending, hard) / `ScheduleAnyway`(그래도 배치, soft) 둘 중 하나예요.

복습 스케줄러는 어떻게 노드를 고를까?

고급 기능을 배우기 전에, 스케줄러의 2단계 의사결정을 짚고 갑니다. (11장 스케줄러 복습)

🎯 **한 줄 결론:** 스케줄러는 새 파드(아직 `nodeName`이 비어있는 파드)를 보면 ① **필터링(Filtering)**으로 "들어갈 수 있는 노드"를 추리고 → ② **스코어링(Scoring)**으로 "그 중 가장 좋은 노드"에 점수를 매겨 1등을 고른 뒤, `nodeName`을 채워 넣습니다(Bind).

🍔 **쉽게 말하면:** 채용 면접이예요. 지원자(파드)가 오면 ① 먼저 **서류 합격/불합격**(필터링) — 자격 조건 못 맞추면 바로 탈락. ② 살아남은 사람들끼리 **점수 매겨 1등 뽑기**(스코어링). 그리고 합격 통보(Bind). 이 장에서 배우는 모든 규칙은 결국 ①서류전형을 뚝세게 하거나(필수=hard) ②가산점을 주는(선호=soft) 두 가지 중 하나예요.



스케줄러의 2단계 결정 — 이 장의 모든 기능은 ①필터링(hard) 또는 ②스코어링(soft)에 끼어듭니다

hard 규칙 = 필터링에 작용

- ▶ 안 맞으면 탈락 — 후보에서 제외
- ▶ `taint(NoSchedule)`, `required ...` 어피니티
- ▶ 만족하는 노드가 0개면 → 파드 **Pending**

soft 규칙 = 스코어링에 작용

- ▶ 안 맞아도 OK — 점수만 깎임(선호)
- ▶ `preferred ...` 어피니티, 분산 제약 (ScheduleAnyway)
- ▶ 다 안 맞으면 그래도 **어딘가엔 배치**

2판 변화

"predicate / priority" → "Filter / Score 플러그인"으로 용어가 바뀜

1판은 필터링 단계를 **predicate**(서술자), 스코어링 단계를 **priority**(우선순위 함수)라고 불렀습니다. 최신 쿠버네티스는 스케줄링 프레임워크로 재설계되면서 이 둘이 각각 **Filter** 플러그인 / **Score** 플러그인으로 바뀌었어요. 개념(2단계)은 동일하지만 내부가 플러그인화됐습니다. → 슬라이드 11에서 자세히.

이 슬라이드의 용어 풀이

스케줄러

"이 파드를 어느 노드에서 실행할까?"만 결정하는 컴포넌트예요. 파드를 직접 만들거나 실행하지는 않고, 조건에 맞는 노드를 골라 nodeName을 채워줘요.

스케줄링 프레임워크

스케줄러 내부를 여러 "확장 지점"(Filter, Score, PostFilter, Bind 등)으로 쪼개고, 각 지점에 플러그인을 끼워 동작을 바꾸는 최신 구조예요. 예전의 predicate/priority를 대체했어요.

nodeName

파드 스펙에 노드 이름을 직접 박아 스케줄러를 건너뛰고 그 노드에 강제 배치하는 필드예요. 리소스·taint·어피니티를 전부 무시해서 위험해요.

nodeSelector

파드에 "이 레이블을 가진 노드에만 가라"고 적는 가장 단순한 방법이에요. 정확히 일치(AND)만 되고, 노드 어피니티가 그 상위호환이에요.

NoSchedule

taint 효과 중 하나. 건디지 못하는 파드를 그 노드에 "새로 배치하지 않음"이에요. 이미 떠 있는 파드는 건드리지 않아요.

taint

영어로 "오염·열록"이라는 뜻이에요. 노드에 붙이는 "꺼림칙한 딱지(출입 통제 표식)"라고 보면 돼요. 이 딱지를 "괜찮아"라고 받아주는 toleration이 없는 파드는 그 노드에 배치되지 못해요.

16.1 테인트 & 톨러레이션 — 노드가 파드를 "밀어내기"

지금까지의 도구는 전부 "파드를 노드로 끌어당기는" 방향이었어요. 테인트는 반대로 **노드 쪽에서 파드를 막는** 유일한 도구입니다.

🔴 **한 줄 결론:** 테인트는 노드에 붙이는 "출입 통제 표식"이에요. 표식이 붙은 노드는 아무 파드나 못 들어와요. 그 표식을 톨러레이션 (= 그 표식을 "**괜찮아, 참고 들어갈게**"라고 받아주는 자격)으로 가진 파드만 통과할 수 있습니다. **방향이 반대**인 게 핵심 — 어피니티는 "파드가 고르고", 테인트는 "노드가 거부"해요.

💡 **왜 하필 "견딘다(toleration)"는 말을 쓸까?** **taint**의 영어 뜻이 바로 "**오염시키다 · 얼룩**"이에요. 즉 노드에 **taint**를 붙인다 = 그 노드에 "**꺼림칙한 딱지**"를 묻혀 놓는다는 뜻. 보통 파드들은 그 딱지가 싫어서 그 노드를 **피해요** (그래서 기본이 "거부"). 그 노드에 들어가려면 "나는 그 딱지 **괜찮아, 참을 수 있어**"라고 선언해야 하는데, 그 선언이 바로 **toleration**이에요. "**견딘다 = 그 노드의 꺼림칙한 조건을 참아낸다**"로 읽으면 딱 맞아요.

🍔 **쉽게 말하면 (1): 흡연 구역**이에요 🚬. 노드(방)에 "담배 냄새"라는 딱지(taint)가 묻어 있어요. 냄새 싫은 사람(보통 파드)은 안 들어가지만, "난 냄새 **괜찮아**"라고 **참을 수 있는 사람**(toleration 가진 파드)은 들어갈 수 있죠. 여기서 "참는다 = 견딘다"예요.

🍔 **쉽게 말하면 (2): 클럽 입구의 드레스 코드**로도 볼 수 있어요. 노드(클럽)가 "정장 아니면 입장 불가(taint)"를 내걸면, 정장 입은 손님(toleration 가진 파드)만 들어가요. 단, 정장을 입었다고 **꼭 이 클럽에 가야 하는 건 아니예요** — 들어갈 "자격"이 생길 뿐이죠. (전용 노드에 진짜로 묶으려면 어피니티를 같이 써야 함!)

테인트의 3요소: **key=value:effect**

```
# 노드에 테인트 추가 (kubectl)
$ kubectl taint nodes node1 node-type=production:NoSchedule
      |key|  |value|  |effect|
```

<u>effect</u> (효과)	새로 들어오려는 파드	이미 실행 중인 파드
NoSchedule	못 견디면(=toleration 없으면) 배치 거부 (hard)	건드리지 않음 (그대로 유지)
PreferNoSchedule	되도록 피함, 자리 없으면 허용 (soft)	건드리지 않음
NoExecute	견디지 못하면 배치 거부	견디지 못하면 쫓아냄(축출) ⚠️

자동 테인트

마스터 노드는 왜 파드가 안 뜰까?

컨트롤 플레인 노드엔 자동으로 taint가 붙어 일반 파드를 막아요. node.kubernetes.io/* 자동 테인트도 정리.

컨트롤 플레인 노드

kubeadm으로 만든 클러스터의 컨트롤 플레인 노드에는 자동으로 `node-`

`role.kubernetes.io/control-plane:NoSchedule` taint가 붙어 일반 파드를 막습니다. (1판의 `master` 키는 1.25에서 제거됨)

노드 상태 기반 (자동)

- `node.kubernetes.io/not-ready` — 노드가 준비 안 됨
- `node.kubernetes.io/unreachable` — 노드와 통신 끊김
- `node.kubernetes.io/memory-pressure, disk-pressure, pid-pressure, unschedulable`

이들은 보통 `NoExecute` 또는 `NoSchedule`로 붙어, 문제 노드에서 파드를 자동 축출/차단합니다 (TaintBasedEvictions).

NoExecute

tolerationSeconds — "몇 초까지만 버티기"

노드가 not-ready가 되면 일정 시간 후 파드를 축출. 그 유예 시간을 조절하는 법.

`NoExecute` taint가 붙은 노드에서, toleration에 `tolerationSeconds`를 주면 "그 시간 동안만 버티다 축출"됩니다.

```
tolerations:
- key: node.kubernetes.io/unreachable
  operator: Exists
  effect: NoExecute
  tolerationSeconds: 300 # 5분 버티고 축출
```

쿠버네티스는 모든 파드에 not-ready/unreachable에 대한 **기본 5분(300초) toleration**을 자동 추가합니다. 즉 노드가 갑자기 죽어도 5분간은 파드를 옮기지 않고 회복을 기다려요. 더 빠르게/느리게 옮기려면 이 값을 조절합니다.

최신 변화

마스터 노드 taint 키 변경: `master` → `control-plane`

1판은 마스터 노드 taint를 `node-role.kubernetes.io/master:NoSchedule`로 설명합니다. 하지만 포용적 용어 정책에 따라 `node-role.kubernetes.io/control-plane`으로 바뀌었고, K8s 1.25부터 옛 `master` 키는 완전히 제거됐습니다. 또한 노드 상태 기반 자동 축출(TaintBasedEvictions)이 GA되어, not-ready/unreachable 노드의 파드를 taint로 자동 축출합니다.

📖 이 슬라이드의 용어 풀이

축출

실행 중인 파드를 노드에서 강제로 내보내는 것(eviction)이에요. NoExecute taint나 우선순위 선점에서 일어나요.

컨트롤 플레인

클러스터의 "두뇌" 영역이에요. API 서버·etcd·스케줄러·컨트롤러 매니저가 모여 있어요. 예전엔 "마스터 노드"라고 불렀어요.

테인트

노드에 붙이는 "출입 통제 표식"이에요(taint). key=value:effect 형태로, 이 표식을 건디는 toleration이 없는 파드는 그 노드에 못 들어와요. 어피니티와 반대로 "밀어내는" 방향이에요.

톨러레이션

파드가 노드의 taint(꺼림칙한 딱지)를 "괜찮아, 참고 들어갈게"라고 받아주는 표식이에요(toleration). "견딘다"는 = "그 딱지를 참아낸다"는 뜻. taint와 짝이 맞아야 그 노드에 들어갈 "자격"이 생겨요. 단, 꼭 그 노드로 간다는 뜻은 아니에요.

effect

taint가 파드에 미치는 효과예요. NoSchedule(새 배치 금지) / PreferNoSchedule(되도록 금지) / NoExecute(실행 중인 것도 축출) 세 가지가 있어요.

NoExecute

taint 효과 중 하나. 새 파드 배치를 막는 건 물론, "이미 실행 중인데 견디지 못하는 파드까지 쫓아내요(축출)". 노드 장애 시 자동 축출에 쓰여요.

NoSchedule

taint 효과 중 하나. 견디지 못하는 파드를 그 노드에 "새로 배치하지 않음"이에요. 이미 떠 있는 파드는 건드리지 않아요.

PreferNoSchedule

taint 효과 중 하나. "되도록 배치하지 말되, 다른 자리가 없으면 허용"하는 약한(soft) 버전이에요.

taint

영어로 "오염·얼룩"이라는 뜻이에요. 노드에 붙이는 "꺼림칙한 딱지(출입 통제 표식)"라고 보면 돼요. 이 딱지를 "괜찮아"라고 받아주는 toleration이 없는 파드는 그 노드에 배치되지 못해요.

TaintBasedEvictions

노드가 not-ready/unreachable가 되면 자동으로 NoExecute taint를 붙여, 일정 시간 후 그 노드의 파드를 다른 노드로 옮기는(축출) 메커니즘이에요. 최신 버전에서 표준 동작이에요.

toleration

파드가 노드의 taint를 "괜찮아, 참을 수 있어"라고 받아주는 표식이에요. "견딘다 = 그 노드의 꺼림칙한 조건을 참아낸다". taint와 key/effect가 맞으면 그 노드에 들어갈 자격이 생겨요.

tolerationSeconds

NoExecute taint를 "몇 초까지 견디다 축출될지" 정하는 값이에요. 노드가 죽어도 기본 300초(5분)는 파드를 옮기지 않고 회복을 기다려요.

16.1 실습 — taint 붙이고 toleration 매칭하기

전용 노드(production 전용)를 만들어, 거기엔 production 파드만 들어가게 해봅니다.

① 노드에 테인트

```
# node1을 production 전용으로
$ kubectl taint nodes node1 \
    node-type=production:NoSchedule

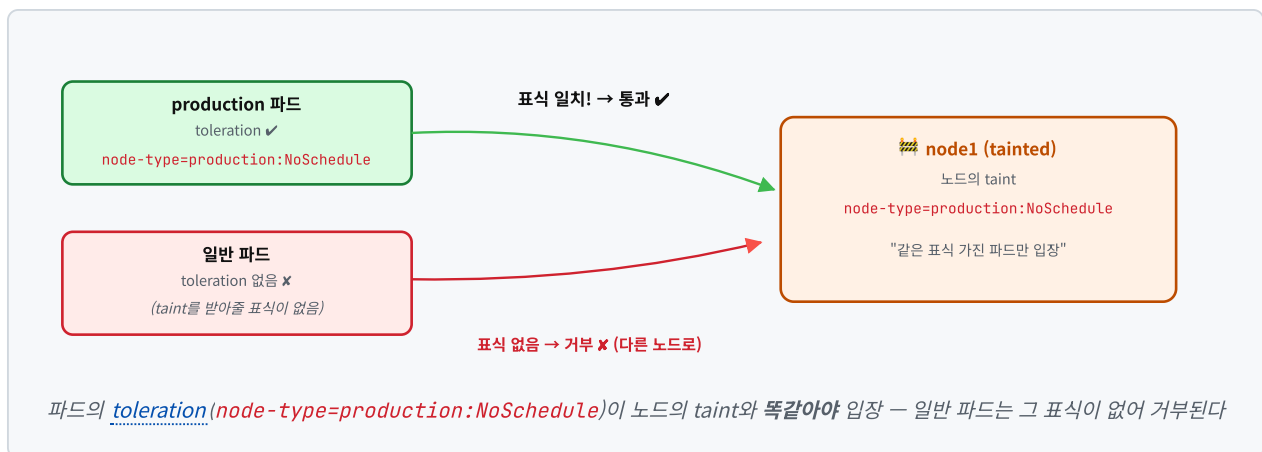
# 확인
$ kubectl describe node node1 | grep Taint
Taints: node-type=production:NoSchedule

# 테인트 제거 (끝에 - 붙이기)
$ kubectl taint nodes node1 \
    node-type=production:NoSchedule-
```

② 파드에 톨러레이션

```
apiVersion: apps/v1
kind: Deployment
spec:
  template:
    spec:
      tolerations:
        - key: node-type
          operator: Equal # 또는 Exists
          value: production
          effect: NoSchedule
```

operator 두 가지: **Equal**(key+value 둘 다 일치) / **Exists**(key만 있으면 됨, value 생략). key까지 생략하고 **Exists**만 두면 **모든 taint를 건딤**(예: **DaemonSet**).



⚠ **흔한 오해:** toleration은 "이 노드에 갈 수 있는 자격"일 뿐, "이 노드에 가야 한다"가 아니에요. production 파드가 toleration만 있고 다른 taint 없는 노드도 비어 있으면 거기로 갈 수도 있어요. "전용 노드에 묶기"는 **taint(밀어내기)** + **nodeAffinity/nodeSelector(끌어당기기)**를 같이 써야 완성됩니다.

이 슬라이드의 용어 풀이

테인트

노드에 붙이는 "출입 통제 표식"이에요(taint). key=value:effect 형태로, 이 표식을 건디는 toleration이 없는 파드는 그 노드에 못 들어와요. 어피니티와 반대로 "밀어내는" 방향이에요.

톨러레이션

파드가 노드의 taint(꺼림칙한 딱지)를 "괜찮아, 참고 들어갈게"라고 받아주는 표식이에요(toleration). "견딘다"는 = "그 딱지를 참아낸다"는 뜻. taint와 짝이 맞아야 그 노드에 들어갈 "자격"이 생겨요. 단, 꼭 그 노드로 간다는 뜻은 아니에요.

DaemonSet

클러스터의 모든(또는 일부) 노드마다 파드를 하나씩 띄우는 리소스예요. 로그 수집·모니터링·네트워크 에이전트처럼 노드별로 필요한 것에 써요.

effect

taint가 파드에 미치는 효과예요. NoSchedule(새 배치 금지) / PreferNoSchedule(되도록 금지) / NoExecute(실행 중인 것도 축출) 세 가지가 있어요.

nodeSelector

파드에 "이 레이블을 가진 노드에만 가라"고 적는 가장 단순한 방법이에요. 정확히 일치(AND)만 되고, 노드 어피니티가 그 상위호환이에요.

NoSchedule

taint 효과 중 하나. 건디지 못하는 파드를 그 노드에 "새로 배치하지 않음"이에요. 이미 떠 있는 파드는 건드리지 않아요.

taint

영어로 "오염·얼룩"이라는 뜻이에요. 노드에 붙이는 "꺼림칙한 딱지(출입 통제 표식)"라고 보면 돼요. 이 딱지를 "괜찮아"라고 받아주는 toleration이 없는 파드는 그 노드에 배치되지 못해요.

toleration

파드가 노드의 taint를 "괜찮아, 참을 수 있어"라고 받아주는 표식이에요. "견딘다 = 그 노드의 꺼림칙한 조건을 참아낸다". taint와 key/effect가 맞으면 그 노드에 들어갈 자격이 생겨요.

16.1.4 테인트 & 톨러레이션 — 언제 쓰나?

실무에서 [taint](#)가 빛나는 대표 시나리오 4가지

사례 1

전용 노드 (Dedicated Nodes)

특정 팀/고객/워크로드만 쓰는 노드. 아무 파드나 못 들어오게 막고, 허가된 파드만 [toleration](#)으로 입장.

특정 팀이나 고객 전용 노드를 만들고 싶을 때:

1. 노드에 taint: `kubectl taint nodes node1 dedicated=team-a:NoSchedule`
2. 허가된 파드에만 toleration 추가
3. + `nodeAffinity`로 그 파드를 해당 노드로 유치 (toleration만으로는 다른 노드로 쉼 수 있음)

taint(차단) + affinity(유치) = 진짜 전용 노드

사례 2

특수 하드웨어 노드 (GPU 등)

비싼 GPU 노드에 일반 파드가 끼어들어 자리를 낭비 하지 않도록 taint로 보호.

GPU 노드는 비싸므로, GPU가 필요 없는 일반 파드가 끼어들어 자리를 차지하면 낭비입니다.

- GPU 노드에 taint: `gpu=true:NoSchedule`
- GPU 파드만 toleration + `nodeAffinity(gpu In [true])`

최신에선 GPU 같은 장치 할당에 **DRA(동적 리소스 할당)**를 함께 고려할 수 있습니다.

사례 3

노드 장애 시 [축출 \(NoExecute\)](#)

노드가 not-ready/unreachable가 되면 자동 taint가 붙고, 일정 시간 후 파드를 다른 노드로 옮김.

노드가 not-ready/unreachable가 되면 컨트롤러가 **NoExecute** taint를 자동으로 붙입니다. 파드는 기본 toleration(300초)이 끝나면 축출되어 건강한 노드로 재스케줄됩니다.

이게 "노드가 죽으면 파드가 자동으로 다른 곳에서 살아나는" 메커니즘의 일부예요(TaintBasedEvictions).

사례 4

시스템 파드만 허용

[DaemonSet](#)(로그·모니터링 에이전트)은 모든 taint를 건디게 해서 마스터·전용 노드에도 빠짐없이 배치.

로그 수집기·모니터링 에이전트(DaemonSet)는 마스터·전용 노드를 포함한 **모든 노드**에 떠야 합니다. 그래서 이런 파드는 "모든 taint를 건디는" toleration을 담니다:

```
tolerations:
- operator: Exists # key/effect 생략 = 모든 taint 허용
```

실제로 kube-proxy, CNI, 로깅 DaemonSet들이 이 패턴을 씁니다.

	테인트/톨러레이션 (16.1)	노드 어피니티 (16.2)
방향	노드가 파드를 밀어냄 (repel)	파드가 노드에 끌림 (attract)
주체	노드(클러스터 관리자)가 설정	파드(앱 개발자)가 설정
기본 동작	아무 파드나 거부 가 기본	지정 안 하면 아무 노드나 OK

"전용 노드"	다른 파드를 쫓아내는 역할	내 파드를 그 노드로 끌어오는 역할
---------	----------------	---------------------

💡 **핵심:** 둘은 경쟁 관계가 아니라 짝꿍이에요. 진짜 "전용 노드"를 만들려면 **taint(다른 파드 차단) + nodeAffinity(내 파드 유치)**를 함께 써야 완성됩니다.

📖 이 슬라이드의 용어 풀이

노드 어피니티

파드가 "이런 레이블을 가진 노드에 가고 싶다"를 표현하는 규칙이에요. nodeSelector보다 표현력이 좋아 In/NotIn 등 연산자와 필수/선택 두 단계를 지원해요.

축출

실행 중인 파드를 노드에서 강제로 내보내는 것(eviction)이에요. NoExecute taint나 우선순위 선점에서 일어나요.

테인트

노드에 붙이는 "출입 통제 표식"이에요(taint). key=value:effect 형태로, 이 표식을 건디는 toleration이 없는 파드는 그 노드에 못 들어와요. 어피니티와 반대로 "밀어내는" 방향이에요.

톨러레이션

파드가 노드의 taint(꺼림칙한 딱지)를 "괜찮아, 참고 들어갈게"라고 받아주는 표식이에요(toleration). "견딘다"는 = "그 딱지를 참아낸다"는 뜻. taint와 짝이 맞아야 그 노드에 들어갈 "자격"이 생겨요. 단, 꼭 그 노드로 간다는 뜻은 아니에요.

DaemonSet

클러스터의 모든(또는 일부) 노드마다 파드를 하나씩 띄우는 리소스예요. 로그 수집·모니터링·네트워크 에이전트처럼 노드별로 필요한 것에 써요.

DRA

Dynamic Resource Allocation. GPU 같은 특수 장치를 기존 방식보다 유연하게 요청·할당하는 최신 메커니즘이에요. AI/ML 워크로드에서 주목받아요.

effect

taint가 파드에 미치는 효과예요. NoSchedule(새 배치 금지) / PreferNoSchedule(되도록 금지) / NoExecute(실행 중인 것도 축출) 세 가지가 있어요.

NoExecute

taint 효과 중 하나. 새 파드 배치를 막는 건 물론, "이미 실행 중인데 견디지 못하는 파드까지 쫓아내요(축출)". 노드 장애 시 자동 축출에 쓰여요.

NoSchedule

taint 효과 중 하나. 견디지 못하는 파드를 그 노드에 "새로 배치하지 않음"이에요. 이미 떠 있는 파드는 건드리지 않아요.

taint

영어로 "오염·얼룩"이라는 뜻이에요. 노드에 붙이는 "꺼림칙한 딱지(출입 통제 표식)"라고 보면 돼요. 이 딱지를 "괜찮아"라고 받아주는 toleration이 없는 파드는 그 노드에 배치되지 못해요.

TaintBasedEvictions

노드가 not-ready/unreachable가 되면 자동으로 NoExecute taint를 붙여, 일정 시간 후 그 노드의 파드를 다른 노드로 옮기는(축출) 메커니즘이에요. 최신 버전에서 표준 동작이에요.

toleration

파드가 노드의 taint를 "괜찮아, 참을 수 있어"라고 받아주는 표식이에요. "견딘다 = 그 노드의 꺼림칙한 조건을 참아낸다". taint와 key/effect가 맞으면 그 노드에 들어갈 자격이 생겨요.

16.2 노드 어피니티 — 파드를 특정 노드로 "끌어당기기"

`nodeSelector`(3장)의 강력한 상위호환. 단순 일치를 넘어 "이런 노드 중 하나" 같은 표현이 가능해요.

☞ **한 줄 결론:** 노드 어피니티는 노드의 레이블을 보고 "내 파드가 갈 노드"의 조건을 거는 규칙이에요.

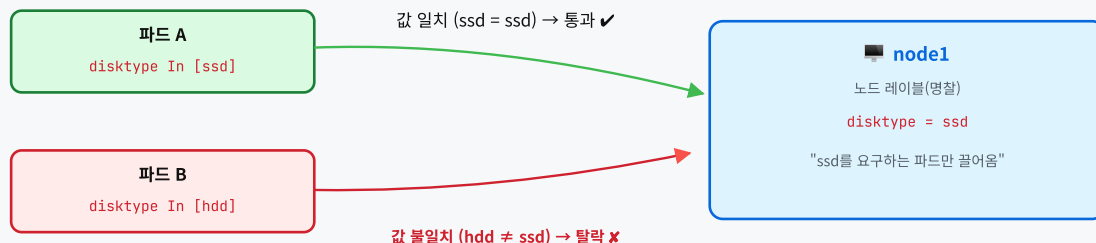
`nodeSelector`가 "`disktype=ssd` 정확히 일치"만 됐다면, 노드 어피니티는 `In / NotIn / Exists / Gt / Lt` 같은 연산자와 `hard(필수) / preferred(선호)` 두 단계를 지원합니다.

🍔 **쉽게 말하면:** 방금 배운 **테인트**가 "노드가 파드를 밀어내기"였다면, 어피니티는 정반대로 "파드가 노드한테 끌리기"예요. 이사할 동네 **고르기**라고 보면 돼요. 🏠 — 집(노드)마다 명찰(레이블)이 붙어 있어요(`disktype=ssd`, `gpu=true`, `zone=seoul`). 파드가 "나는 **ssd 명찰** 단 집으로 갈래!" 하고 자기 갈 곳을 직접 고르는 거예요. 그게 노드 어피니티입니다.

	<code>nodeSelector</code> (구식, 1판 3장)	<code>nodeAffinity</code> (16.2)
표현력	레이블 정확 일치(AND)만	<code>In/NotIn/Exists/Gt/Lt</code> 연산자
강도	무조건 <code>hard</code>	<code>hard</code> + <code>preferred</code> (가중치) 둘 다
문법	한 줄, 간단	장황하지만 유연
현재 위상	여전히 유효(간단할 땐 OK)	복잡한 조건의 표준

핵심 원리 — 파드가 요구한 값 = 노드 레이블 값이 "같아야" 매칭

노드 어피니티가 동작하려면, **노드에 붙은 레이블 값과 파드가 어피니티로 요구한 값이 똑같아야** 해요. 노드가 `disktype=ssd`인데 파드가 `disktype In [hdd]`를 요구하면 값이 달라 → 그 노드는 후보에서 탈락합니다. (`taint/toleration`이 `key-value-effect`를 맞춰야 했던 것과 똑같은 "짜 맞추기" 원리예요.)



파드가 요구한 값(ssd)과 노드 레이블 값(ssd)이 같아야 매칭 — 다른 값(hdd)을 요구하면 그 노드는 후보에서 빠진다

긴 이름의 비밀: `requiredDuringSchedulingIgnoredDuringExecution`

requiredDuringScheduling

배치할 때(스케줄링) = 필수/선호
required(hard) ↔ preferred(soft)

+

IgnoredDuringExecution

이미 실행 중이면 = 무시
나중에 노드 레이블이 바뀌어도 축출 안 함

이름 = "배치할 때의 강도" + "실행 중 레이블 변화 처리" 두 부분의 조합

참고

RequiredDuringExecution 은 아직도 미구현

이름이 **IgnoredDuringExecution** 으로 끝나는 이유는, 언젠가 "실행 중에도 레이블이 안 맞으면 축출하는" **RequiredDuringExecution** 변형을 추가하려고 자리를 비워둔 거예요. 1판 시절부터 계획만 있고 **최신 버전에서** **도 여전히 미구현**입니다. 즉 현재 노드 어피니티는 모두 "배치 순간"에만 작동해요.

이 슬라이드의 용어 풀이

노드 어피니티

파드가 "이런 레이블을 가진 노드에 가고 싶다"를 표현하는 규칙이에요. nodeSelector보다 표현력이 좋아 In/NotIn 등 연산자와 필수/선호 두 단계를 지원해요.

레이블

리소스(노드·파드 등)에 붙이는 key=value 꼬리표예요. 스케줄링 규칙(어피니티·셀렉터)이 이 레이블을 보고 대상을 고릅니다.

축출

실행 중인 파드를 노드에서 강제로 내보내는 것(eviction)이에요. NoExecute taint나 우선순위 선점에서 일어나요.

테인트

노드에 붙이는 "출입 통제 표식"이에요(taint). key=value:effect 형태로, 이 표식을 건디는 toleration이 없는 파드는 그 노드에 못 들어와요. 어피니티와 반대로 "밀어내는" 방향이에요.

effect

taint가 파드에 미치는 효과예요. NoSchedule(새 배치 금지) / PreferNoSchedule(되도록 금지) / NoExecute(실행 중인 것도 축출) 세 가지가 있어요.

IgnoredDuringExecution

"이미 실행 중이면 무시"라는 뜻이에요. 배치된 뒤 노드 레이블이 바뀌어 조건이 안 맞아도 파드를 쫓아내지 않아요. (실행 중 축출하는 RequiredDuringExecution은 아직 미구현)

nodeSelector

파드에 "이 레이블을 가진 노드에만 가라"고 적는 가장 단순한 방법이에요. 정확히 일치(AND)만 되고, 노드 어피니티가 그 상위호환이에요.

requiredDuringScheduling

"배치할 때 반드시 만족해야 하는" 필수(hard) 규칙이에요. 맞는 노드가 하나도 없으면 파드는 Pending 상태로 멈춰요.

taint

영어로 "오염·열록"이라는 뜻이에요. 노드에 붙이는 "꺼림칙한 딱지(출입 통제 표식)"라고 보면 돼요. 이 딱지를 "괜찮아"라고 받아주는 toleration이 없는 파드는 그 노드에 배치되지 못해요.

toleration

파드가 노드의 taint를 "괜찮아, 참을 수 있어"라고 받아주는 표식이에요. "견딘다 = 그 노드의 꺼림칙한 조건을 참아낸다". taint와 key/effect가 맞으면 그 노드에 들어갈 자격이 생겨요.

16.2 hard 규칙 vs preferred 규칙

필수 조건(required)과 선호 조건(preferred)을 YAML로 비교

🍔 쉽게 말하면: 집 구할 때 조건 거는 거랑 똑같아요 🏠 — **required(필수)** = "역세권 **아니면 절대 안 살아**" (조건 맞는 집 없으면 차라리 노숙 = 파드 Pending). **preferred(선호)** = "역세권이면 **좋은데**, 없으면 딴 데도 OK" (가산점만 줌). **weight**는 그 가산점의 크기예요 — "역세권 80점, 1층 20점" 처럼 점수를 합쳐서 제일 높은 집을 골라요.

① hard — 필수 (required)

```
affinity:
  nodeAffinity:
    requiredDuringScheduling
    IgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
            - key: gpu
              operator: In
              values: ["true"]
```

조건 맞는 노드가 **하나도 없으면** → **Pending**. "반드시 GPU 노드에만".

② preferred — 선호 (가중치)

```
affinity:
  nodeAffinity:
    preferredDuringScheduling
    IgnoredDuringExecution:
      - weight: 80 # 1~100
        preference:
          matchExpressions:
            - key: zone
              operator: In
              values: ["zone1"]
```

zone1을 **선호하되**, 없으면 다른 곳도 OK. **weight**가 클수록 강하게 끌림.

매칭 연산자 (operator)

연산자	의미	비고
<code>In</code> / <code>NotIn</code>	<u>레이블</u> 값이 목록에 있음 / 없음	가장 많이 씀
<code>Exists</code> / <code>DoesNotExist</code>	키가 있음 / 없음 (값 무관)	values 생략
<code>Gt</code> / <code>Lt</code>	값이 ~보다 큼 / 작음 (정수)	<u>노드 어피니티</u> 전용

2판 변화

여러 `nodeSelectorTerms` 는 **OR**, `matchExpressions` 는 **AND**

헷갈리기 쉬운 논리 규칙: `nodeSelectorTerms` 항목이 여러 개면 **그 중 하나만** 맞으면 됩니다(OR). 반대로 한 term 안의 `matchExpressions` 가 여러 개면 **모두** 맞아야 합니다(AND). 2판은 이 OR/AND 동작과, `matchFields` (레이블이 아닌 `metadata.name` 같은 필드 매칭)를 더 명확히 다룹니다.

이 슬라이드의 용어 풀이

노드 어피니티

파드가 "이런 레이블을 가진 노드에 가고 싶다"를 표현하는 규칙이에요. nodeSelector보다 표현력이 좋아 In/NotIn 등 연산자와 필수/선택 두 단계를 지원해요.

레이블

리소스(노드·파드 등)에 붙이는 key=value 꼬리표예요. 스케줄링 규칙(어피니티·셀렉터)이 이 레이블을 보고 대상을 고릅니다.

IgnoredDuringExecution

"이미 실행 중이면 무시"라는 뜻이에요. 배치된 뒤 노드 레이블이 바뀌어 조건이 안 맞아도 파드를 쫓아내지 않아요. (실행 중 축출하는 RequiredDuringExecution은 아직 미구현)

matchExpressions

레이블 조건을 연산자로 표현하는 방식이에요. key/operator(In·NotIn·Exists·DoesNotExist·Gt·Lt)/values로 유연하게 매칭해요.

nodeSelector

파드에 "이 레이블을 가진 노드에만 가라"고 적는 가장 단순한 방법이에요. 정확히 일치(AND)만 되고, 노드 어피니티가 그 상위호환이에요.

preferredDuringScheduling

"배치할 때 되도록 만족하면 좋은" 선호(soft) 규칙이에요. 못 맞춰도 배치는 되고, weight(1~100)로 선호 강도를 줘요.

requiredDuringScheduling

"배치할 때 반드시 만족해야 하는" 필수(hard) 규칙이에요. 맞는 노드가 하나도 없으면 파드는 Pending 상태로 멈춰요.

weight

preferred(선호) 규칙의 가중치(1~100)예요. 여러 선호 조건의 weight가 합산되어 노드 점수가 되고, 점수가 높은 노드가 선택돼요.

16.3 파드 어피니티 — 다른 "파드" 곁에 두기

노드 어피니티는 "노드 레이블" 기준이었죠. 파드 어피니티는 이미 떠 있는 다른 파드를 기준으로 배치합니다.

🎯 **한 줄 결론:** "이 파드를 저 파드와 같은 곳에 두고 싶다"를 표현해요. 예: 웹 서버를 캐시(Redis) 파드와 같은 노드에 붙여 네트워크 지연을 줄이기. 핵심은 `topologyKey` — "같은 곳"이 같은 노드인지, 같은 존인지, 같은 리전인지를 정하는 다이얼입니다.

🍪 **쉽게 말하면:** "나는 제일 친한 친구(캐시 파드) 옆자리에 앉을래"예요. 그런데 '옆자리'의 범위를 `topologyKey`로 정해요 — 같은 책상(노드)? 같은 교실(존)? 같은 학교(리전)? 범위를 좁힐수록 더 가까이 붙습니다.

`topologyKey`가 "같은 곳"의 범위를 정한다

`kubernetes.io/hostname`

= 같은 "노드"에

가장 가깝게 붙임

지연 최소 / 장애 공유 최대

`topology.kubernetes.io/zone`

= 같은 "가용 영역(존)"에

적당히 가깝게

균형 잡힌 선택

`topology.kubernetes.io/region`

= 같은 "리전"에

느슨하게

지연 큼 / 장애 격리 큼

← 좁음(붙음/빠름/위험공유) ————— 넓음(흩어짐/느림/안전) →

```
affinity:
  podAffinity:           # 곁에 두기 (anti면 멀리)
    requiredDuringSchedulingIgnoredDuringExecution:
      - topologyKey: kubernetes.io/hostname  # "같은 곳" = 같은 노드
        labelSelector:
          matchLabels:
            app: redis      # app=redis 파드가 있는 노드로 끌림
```

16.3.1~2

같은 노드 / 같은 존에 배치

topologyKey만 바꾸면 "같은 노드"에서 "같은 존"까지 확장. 동작 원리 상세.

웹 서버를 캐시와 같은 곳에 두는 예:

```
podAffinity:
  requiredDuringSchedulingIgnoredDuringExecution:
  - labelSelector:
      matchLabels: { app: redis }
    topologyKey: kubernetes.io/hostname
```

- `topologyKey: kubernetes.io/hostname`
→ 같은 노드
- `topologyKey: topology.kubernetes.io/zone` → 같은 존

동작: 스케줄러가 "app=redis 파드가 이미 있는 노드(또는 존)"를 찾아, 그 토폴로지 도메인 안에 새 파드를 배치합니다.

16.3.3

선호로 표현하기 (preferred)

"되도록 같이"지만 자리 없으면 따로. [weight](#) 기반 soft 파드 어피니티.

```
podAffinity:
  preferredDuringSchedulingIgnoredDuringExecution:
  - weight: 80
    podAffinityTerm:
      labelSelector:
        matchLabels: { app: redis }
      topologyKey: kubernetes.io/hostname
```

required와 달리 "redis가 있는 노드가 꼭 찾으면" 다른 노드로도 배치됩니다. weight(1~100)가 클수록 같이 두려는 경향이 강해져요. 여러 preferred 항목의 weight가 합산되어 노드 점수가 됩니다.

이 슬라이드의 용어 풀이

가용 영역

같은 리전 안에서 전력·네트워크가 물리적으로 분리된 데이터센터 단위예요(존, zone). 존을 나눠 배치하면 한 존이 통째로 죽어도 서비스가 살아남아요.

노드 어피니티

파드가 "이런 레이블을 가진 노드에 가고 싶다"를 표현하는 규칙이에요. nodeSelector보다 표현력이 좋아 In/NotIn 등 연산자와 필수/선호 두 단계를 지원해요.

레이블

리소스(노드·파드 등)에 붙이는 key=value 꼬리표예요. 스케줄링 규칙(어피니티·셀렉터)이 이 레이블을 보고 대상을 고릅니다.

스케줄러

"이 파드를 어느 노드에서 실행할까?"만 결정하는 컴포넌트예요. 파드를 직접 만들거나 실행하지는 않고, 조건에 맞는 노드를 골라 nodeName을 채워줘요.

파드 어피니티

노드가 아니라 "다른 파드"를 기준으로 배치하는 규칙이에요. "저 파드가 있는 곳에 같이 뒤"를 표현해요. (반대는 안티 어피니티)

IgnoredDuringExecution

"이미 실행 중이면 무시"라는 뜻이에요. 배치된 뒤 노드 레이블이 바뀌어 조건이 안 맞아도 파드를 쫓아내지 않아요. (실행 중 축출하는 RequiredDuringExecution은 아직 미구현)

preferredDuringScheduling

"배치할 때 되도록 만족하면 좋은" 선호(soft) 규칙이에요. 못 맞춰도 배치는 되고, weight(1~100)로 선호 강도를 줘요.

requiredDuringScheduling

"배치할 때 반드시 만족해야 하는" 필수(hard) 규칙이에요. 맞는 노드가 하나도 없으면 파드는 Pending 상태로 멈춰요.

topologyKey

파드 어피니티에서 "같은 곳"의 범위를 정하는 레이블 키예요. hostname이면 "같은 노드", zone이면 "같은 가용 영역"처럼 도메인 크기를 바꿔요.

weight

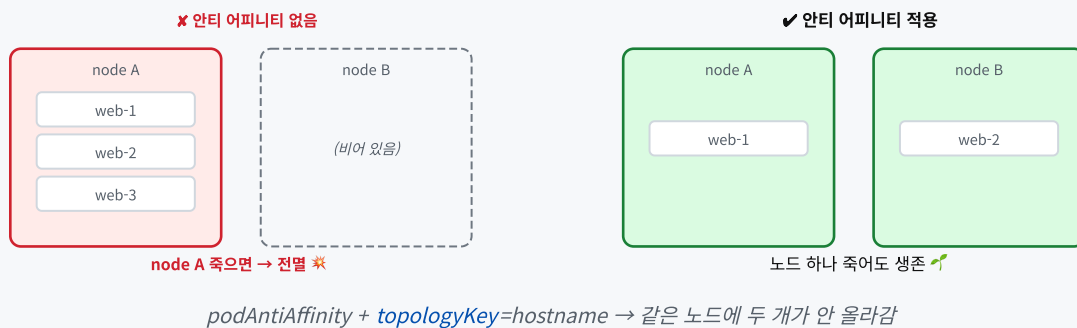
preferred(선호) 규칙의 가중치(1~100)예요. 여러 선호 조건의 weight가 합산되어 노드 점수가 되고, 점수가 높은 노드가 선택돼요.

16.3.4 파드 안티 어피니티 — 서로 "떼어놓기"

어피니티의 반대. 같은 종류의 파드를 일부러 서로 다른 노드/존에 흩뿌려 가용성을 높입니다.

🌀 **한 줄 결론:** 안티 어피니티는 "내 짝(같은 레이블 파드)이 이미 있는 곳은 피해라"예요. 대표 용도: **웹 서버 레플리카 3개를 서로 다른 노드에 1개씩** 배치 → 노드 하나가 죽어도 나머지 2개가 살아남아요. (한 노드에 다 몰려 있으면 그 노드 장애 시 전멸)

🍪 **쉽게 말하면:** "계란을 한 바구니에 담지 마라"예요 🍪. 똑같은 웹 서버 3개를 한 노드(바구니)에 다 몰아두면, 그 노드가 떨어지는 순간 셋 다 깨져요(서비스 전멸). 그래서 "같은 종류 파드가 이미 있는 노드는 피해"라고 해서 일부러 서로 다른 노드에 흩어놓는 거예요. 어피니티(결에 모으기)의 정반대죠.



```
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - topologyKey: kubernetes.io/hostname
        labelSelector:
          matchLabels: { app: web } # "app=web이 이미 있는 노드"는 피함
```

2판 변화 — 중요!

이 용도는 이제 **토폴로지 분산 제약**이 더 낫습니다

"레플리카를 노드/존에 고르게 흩뿌리기"는 1판에선 안티 어피니티가 유일한 답이었어요. 하지만

`requiredDuringScheduling` 안티 어피니티는 "한 노드에 1개"처럼 **경직되고**, 파드가 많아지면 **성능 비용 ($O(N^2)$ 비교)**이 큼니다. 그래서 최신 K8s는 "존마다 **최대 N개 차이**까지 균등하게" 같은 유연한 분산을 위한 `topologySpreadConstraints`를 권장합니다 → 다음 슬라이드.

이 슬라이드의 용어 풀이

레이블

리소스(노드·파드 등)에 붙이는 key=value 꼬리표예요. 스케줄링 규칙(어피니티·셀렉터)이 이 레이블을 보고 대상을 고릅니다.

안티 어피니티

"같은 종류의 파드가 이미 있는 곳은 피해라"는 규칙이에요. 레플리카를 서로 다른 노드/존에 흩뿌려 한 곳이 죽어도 살아남게 해요.

토폴로지 분산 제약

파드를 존/노드에 고르게 퍼 바르는 최신 도구예요(topologySpreadConstraints). maxSkew로 "도메인 간 개수 차이"를 제한해, 안티 어피니티보다 유연하게 균등 분산해요.

IgnoredDuringExecution

"이미 실행 중이면 무시"라는 뜻이에요. 배치된 뒤 노드 레이블이 바뀌어 조건이 안 맞아도 파드를 쫓아내지 않아요. (실행 중 축출하는 RequiredDuringExecution은 아직 미구현)

requiredDuringScheduling

"배치할 때 반드시 만족해야 하는" 필수(hard) 규칙이에요. 맞는 노드가 하나도 없으면 파드는 Pending 상태로 멈춰요.

topologyKey

파드 어피니티에서 "같은 곳"의 범위를 정하는 레이블 키예요. hostname이면 "같은 노드", zone이면 "같은 가용 영역"처럼 도메인 크기를 바꿔요.

★ 신규 토폴로지 분산 제약 (Topology Spread Constraints)

1판에 없던 주제 · K8s 1.19 GA · [안티 어피니티](#)의 "균등 분산" 용도를 깔끔하게 대체

☞ **한 줄 결론:** "파드를 존/노드에 **고르게 퍼 발라라**"를 직접 표현하는 도구예요. 핵심 다이얼은 `maxSkew` — "가장 많은 곳과 가장 적은 곳의 **개수 차이**를 N 이하로 유지"라는 뜻. 안티 어피니티가 "같은 곳에 2개 금지(0 or 1)"처럼 흑백이었다면, 분산 제약은 "차이 1까지 허용" 같은 **유연한 균형**이 가능해요.

🍪 **쉽게 말하면:** 반 편성이에요 🏫. 학생(파드)을 1·2·3반(존)에 나눠 넣는데, "어느 반은 30명, 어느 반은 5명" 이렇게 들쭉날쭉하면 안 되죠. `maxSkew: 1` = "반끼리 인원 차이 1명까지만 허용"이라는 뜻이에요. 그래서 10명이면 4-3-3 처럼 최대한 고르게 나눠줘요. 안티 어피니티는 "한 반에 2명 금지"처럼 뽀빠했다면, 이건 "차이만 적으면 OK"라 훨씬 부드러워요.

4가지 핵심 필드

필드	의미
맥스스큐 (<code>maxSkew</code>)	도메인 간 파드 개수 차이의 허용 최대치 (작을수록 더 뽀빠하게 균 등)
토폴로지키 (<code>topologyKey</code>)	분산 기준 도메인 (예: <code>topology.kubernetes.io/zone</code> = 존 별로)
웬 언새티스파이어블 (<code>whenUnsatisfiable</code>)	<code>DoNotSchedule</code> (하드, 못 맞추면 Pending) / <code>ScheduleAnyway</code> (소프트, 점수만)
레이블 셀렉터 (<code>labelSelector</code>)	"누구를" 세어 분산할지 (보통 이 앱의 파드들)

```
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
  labelSelector:
    matchLabels: { app: web }
# → 모든 존의 web 파드 개수 차이를 1 이하로 유지 (예: 3존에 2-2-2, 최악 2-2-1)
```

안티 어피니티 대비 장점

더 새로운 옵션 (1.25+)

- ▶ **유연한 균형** — "1개 차이까지 허용" 표현 가능
- ▶ `minDomains` — 최소 도메인 개수 강제

- ▶ **존 단위** 균등 분산이 자연스러움
- ▶ 큰 클러스터에서 **성능 부담 적음**

- ▶ `nodeAffinityPolicy` / `nodeTaintsPolicy`
- ▶ `matchLabelKeys` — 롤아웃 시 버전별 분리 계산

클러스터 기본값

사실은 이미 적용되고 있다

최신 스케줄러에는 클러스터 수준 기본 분산 제약(`PodTopologySpread` 플러그인의 기본 정책)이 내장돼 있어, 사용자가 아무것도 안 써도 노드/존에 어느 정도 자동으로 퍼 발라줍니다. 명시적으로 더 강하게 통제하고 싶을 때 위 YAML을 추가하는 거예요.

이 슬라이드의 용어 풀이

레이블

리소스(노드·파드 등)에 붙이는 key=value 꼬리표예요. 스케줄링 규칙(어피니티·셀렉터)이 이 레이블을 보고 대상을 고릅니다.

스케줄러

"이 파드를 어느 노드에서 실행할까?"만 결정하는 컴포넌트예요. 파드를 직접 만들거나 실행하지는 않고, 조건에 맞는 노드를 골라 nodeName을 채워줘요.

안티 어피니티

"같은 종류의 파드가 이미 있는 곳은 피해라"는 규칙이에요. 레플리카를 서로 다른 노드/존에 흩뿌려 한 곳이 죽어도 살아남게 해요.

토폴로지 분산 제약

파드를 존/노드에 고르게 퍼 바르는 최신 도구예요(topologySpreadConstraints). maxSkew로 "도메인 간 개수 차이"를 제한해, 안티 어피니티보다 유연하게 균등 분산해요.

maxSkew

토폴로지 분산 제약에서 "가장 많은 도메인과 가장 적은 도메인의 파드 개수 차이" 허용 최대치예요. 작을수록 더 뽁뽁하게 균등해져요.

topologyKey

파드 어피니티에서 "같은 곳"의 범위를 정하는 레이블 키예요. hostname이면 "같은 노드", zone이면 "같은 가용 영역"처럼 도메인 크기를 바꿔요.

whenUnsatisfiable

분산 제약을 못 맞출 때 어떻게 할지예요. DoNotSchedule(못 맞추면 Pending, hard) / ScheduleAnyway(그래도 배치, soft) 둘 중 하나예요.

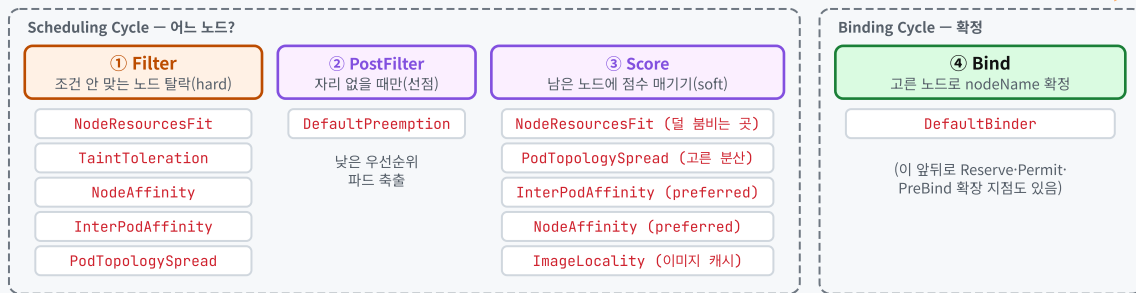
★ 신규 스케줄링 프레임워크 & 프로파일

1판에 없던 주제 · K8s 1.19 GA · [스케줄러](#) 내부가 "플러그인"으로 재설계됨

🎯 **한 줄 결론:** 옛 스케줄러는 필터링(predicate)·스코어링(priority) 로직이 **코드에 박혀** 있어 커스터마이징이 어려웠어요. [스케줄링 프레임워크](#)는 스케줄링 과정을 여러 **확장 지점(extension point)**으로 쪼개고, 각 지점에 **플러그인**을 끼우는 구조로 바꿨습니다. 우리가 쓴 [taint](#)·어피니티·분산 제약도 전부 **내장 플러그인**이에요.

🍌 **쉽게 말하면:** 스마트폰이 **앱(플러그인)**을 **깔 수 있게** 바뀐 거예요 📱. 옛날 폰은 기능이 공장에서 다 박혀 나와서 못 바꿨죠(predicate/priority). 지금 폰은 앱스토어에서 원하는 앱만 골라 깔잖아요 — 스케줄러도 똑같이, 정해진 **꽃는 자리(확장 지점)**에 원하는 **플러그인**만 끼우면 동작이 바뀌어요. [taint](#)·어피니티도 사실 기본 탑재된 "앱"이었던 거예요.

파드 1개를 배치하는 흐름 →



각 확장 지점(① Filter ② PostFilter ③ Score ④ Bind)에 등록된 내장 플러그인들 — 우리가 쓴 [taint](#)·어피니티·분산 제약이 어디서 일하는지 한눈에 (PreFilter·PreScore·Reserve·Permit 등 일부 지점은 생략)

💡 **같은 플러그인이 두 군데 등록돼요:** [TaintToleration](#)·[NodeAffinity](#)·[PodTopologySpread](#) 같은 플러그인은 **Filter(필수 검사)**에서 "들어갈 수 있는 노드냐?"를 거르고, **Score(점수)**에서 다시 "그 중 어디가 더 좋냐?"로 가산점을 줍니다. 즉 하나의 플러그인이 **hard 단계와 soft 단계 양쪽**에서 일해요.

플러그인

익숙한 기능 = 내장 플러그인

TaintToleration, NodeAffinity, InterPodAffinity, PodTopologySpread, NodeResourcesFit 등.

- **TaintToleration** — 16.1 테인트/톨러레이션
- **NodeAffinity** — 16.2 노드 어피니티/셀렉터
- **InterPodAffinity** — 16.3 파드 (안티)어피니티
- **PodTopologySpread** — 토폴로지 분산 제약
- **NodeResourcesFit** — CPU/메모리 적합성
- **DefaultPreemption** — 우선순위 선점 (PostFilter)

각 플러그인은 자신이 관여하는 확장 지점(Filter/Score 등)에 등록되어 동작합니다.

프로파일

스케줄링 프로파일 (여러 개)

한 스케줄러가 여러 프로파일을 운용. 파드의 **schedulerName**으로 골라 쓰기.

예전엔 다른 스케줄링 동작이 필요하면 별도 스케줄러 프로세스를 띄웠어요. 이제는 **KubeSchedulerConfiguration**에 여러 **프로파일**을 정의하고, 각 프로파일에 다른 플러그인 조합/가중치를 줄 수 있습니다.

```
profiles:
- schedulerName: default-scheduler
- schedulerName: high-throughput
plugins: { ... }
```

파드는 **spec.schedulerName**으로 원하는 프로파일을 골라 씁니다.

왜 바뀌었나

"커스텀 스케줄러를 통째로 만들 필요가 없어졌다"

1판 시절 특수한 스케줄링 로직이 필요하면 **스케줄러를 통째로 새로 작성**하거나 여러 스케줄러를 동시에 띄워야 했습니다. 이제는 프레임워크에 **플러그인 하나만 추가**하고 **프로파일**로 조합하면 돼서, 확장이 훨씬 쉬워졌어요.

이 슬라이드의 용어 풀이

노드 어피니티

파드가 "이런 레이블을 가진 노드에 가고 싶다"를 표현하는 규칙이에요. nodeSelector보다 표현력이 좋아 In/NotIn 등 연산자와 필수/선택 두 단계를 지원해요.

선점

높은 우선순위 파드가 배치될 자리가 없을 때, 스케줄러가 낮은 우선순위 파드를 축출해 자리를 만드는 것이에요(preemption).

스케줄러

"이 파드를 어느 노드에서 실행할까?"만 결정하는 컴포넌트예요. 파드를 직접 만들거나 실행하지는 않고, 조건에 맞는 노드를 골라 nodeName을 채워줘요.

스케줄링 프레임워크

스케줄러 내부를 여러 "확장 지점"(Filter, Score, PostFilter, Bind 등)으로 쪼개고, 각 지점에 플러그인을 끼워 동작을 바꾸는 최신 구조예요. 예전의 predicate/priority를 대체했어요.

축출

실행 중인 파드를 노드에서 강제로 내보내는 것(eviction)이에요. NoExecute taint나 우선순위 선점에서 일어나요.

테인트

노드에 붙이는 "출입 통제 표식"이에요(taint). key=value:effect 형태로, 이 표식을 건디는 toleration이 없는 파드는 그 노드에 못 들어와요. 어피니티와 반대로 "밀어내는" 방향이에요.

토폴로지 분산 제약

파드를 존/노드에 고르게 퍼 바르는 최신 도구예요(topologySpreadConstraints). maxSkew로 "도메인 간 개수 차이"를 제한해, 안티 어피니티보다 유연하게 균등 분산해요.

toler레이션

파드가 노드의 taint(꺼림칙한 딱지)를 "괜찮아, 참고 들어갈게"라고 받아주는 표식이에요(toleration). "견딘다"는 = "그 딱지를 참아낸다"는 뜻. taint와 짝이 맞아야 그 노드에 들어갈 "자격"이 생겨요. 단, 꼭 그 노드로 간다는 뜻은 아니예요.

nodeName

파드 스펙에 노드 이름을 직접 박아 스케줄러를 건너뛰고 그 노드에 강제 배치하는 필드예요. 리소스·taint·어피니티를 전부 무시해서 위험해요.

taint

영어로 "오염·얼룩"이라는 뜻이에요. 노드에 붙이는 "꺼림칙한 딱지(출입 통제 표식)"라고 보면 돼요. 이 딱지를 "괜찮아"라고 받아주는 toleration이 없는 파드는 그 노드에 배치되지 못해요.

★ 신규 파드 우선순위 & 선점 (Priority & Preemption)

1판에 없던 주제 · K8s 1.14 GA · "자리가 없으면 덜 중요한 파드를 쫓아낸다"

☞ **한 줄 결론:** `PriorityClass`로 파드에 **중요도(숫자)**를 매겨요. 높은 우선순위 파드가 스케줄될 자리가 없으면, **스케줄러가 낮은 우선순위 파드를 축출(선점, `preemption`)**해 자리를 만들어 줍니다. "프로덕션 결제 서비스 > 배치 분석 작업" 같은 자원 경쟁의 우선순위를 보장해요.

🍔 **쉽게 말하면:** 응급실 **트리아지**예요. 병상(노드 자원)이 꽉 찼는데 위급 환자(높은 우선순위 파드)가 오면, 경증 환자(낮은 우선순위 파드)를 대기실로 돌리고 자리를 내줍니다.

① `PriorityClass` 정의

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000000 # 클수록 우선
globalDefault: false
preemptionPolicy: PreemptLowerPriority
```

② 파드에 부여

```
spec:
  priorityClassName: high-priority
  containers:
  - ...
```

`preemptionPolicy: Never`로 두면 "우선 배치는 받되 남을 쫓아내진 않음". 선점 없이 줄서기만.

개념	설명
우선순위 값	정수. 시스템 파드는 20억대(<code>system-cluster-critical</code> 등) 예약
선점	높은 파드가 Pending이면, 낮은 파드를 축출해 자리 확보
<code>globalDefault</code>	true면 <code>PriorityClass</code> 없는 파드들의 기본값이 됨 (클러스터당 1개)
PostFilter 단계	선점 로직은 <u>스케줄링 프레임워크</u> 의 <code>PostFilter</code> 플러그인에서 동작

주의

선점은 `PodDisruptionBudget`을 "최대한" 존중하지만 보장은 아님

축출 대상 선점 시 `PDB`(`PodDisruptionBudget`)를 가능하면 지키려 하지만, 높은 우선순위 파드를 위해 불가피하면 `PDB`를 어기고도 축출할 수 있습니다. 또한 우선순위는 스케줄링/축출 순서에만 영향을 줄 뿐, 실행 중 CPU/메모리 분배 비율과는 무관해요(그건 `requests/limits-QoS`의 영역).

이 슬라이드의 용어 풀이

선점

높은 우선순위 파드가 배치될 자리가 없을 때, 스케줄러가 낮은 우선순위 파드를 축출해 자리를 만드는 것인데요(preemption).

스케줄러

"이 파드를 어느 노드에서 실행할까?"만 결정하는 컴포넌트예요. 파드를 직접 만들거나 실행하지는 않고, 조건에 맞는 노드를 골라 nodeName을 채워줘요.

스케줄링 프레임워크

스케줄러 내부를 여러 "확장 지점"(Filter, Score, PostFilter, Bind 등)으로 쪼개고, 각 지점에 플러그인을 끼워 동작을 바꾸는 최신 구조예요. 예전의 predicate/priority를 대체했어요.

축출

실행 중인 파드를 노드에서 강제로 내보내는 것(eviction)이에요. NoExecute taint나 우선순위 선점에서 일어나요.

PDB

PodDisruptionBudget. "한 번에 내려갈 수 있는 파드 수의 하한/상한"을 정해, 자발적 축출 시 서비스가 한꺼번에 죽지 않도록 보호하는 리소스예요.

preemptionPolicy

우선순위 파드가 "남을 쫓아낼지" 정하는 정책이에요. PreemptLowerPriority(쫓아냄, 기본) / Never(우선 배치만 받고 선점은 안 함).

PriorityClass

파드의 중요도(정수 값)를 정의하는 리소스예요. 파드는 priorityClassName으로 이걸 참조하고, 값이 클수록 스케줄링·자원 경쟁에서 우선해요.

★ 최신 꼭 알아둘 변경점 & 최신 동향

1판 예제를 그대로 따라 하면 안 되는 부분 + 알아두면 좋은 신기능

! 1판 예제에서 바뀐 well-known 레이블 (가장 흔한 함정)

1판 (deprecated/제거됨)	최신 (현재 표준)
<code>failure-domain.beta.kubernetes.io/zone</code>	<code>topology.kubernetes.io/zone</code>
<code>failure-domain.beta.kubernetes.io/region</code>	<code>topology.kubernetes.io/region</code>
<code>beta.kubernetes.io/arch</code>	<code>kubernetes.io/arch</code>
<code>beta.kubernetes.io/os</code>	<code>kubernetes.io/os</code>
<code>node-role.kubernetes.io/master</code>	<code>node-role.kubernetes.io/control-plane</code>

발표 포인트: 1판 책의 어피니티/분산 예제에 나오는 `failure-domain.beta...` 레이블을 최신 클러스터에 그대로 쓰면 매칭이 안 돼요. 시연/실습 땀 반드시 `topology.kubernetes.io/...` 로 바꿔야 합니다.

신규

Descheduler (디스케줄러)

스케줄러는 "배치 순간"만 봐요. 시간이 지나 불균형해진 파드를 다시 분산시키는 별도 도구.

스케줄러는 "파드를 처음 배치하는 순간"만 봅니다 (IgnoredDuringExecution). 시간이 지나면 노드 추가/삭제, 파드 종료 등으로 배치가 한쪽으로 쏠릴 수 있어요.

Descheduler는 별도 프로젝트(SIG)로, 정책에 따라 "잘못 놓인" 파드를 **축출**해서, 스케줄러가 더 나은 곳에 다시 배치하도록 유도합니다. 예: 저활용 노드 비우기, 어피니티/분산 제약 위반 파드 정리.

신규

동적 리소스 할당 (DRA)

GPU·특수 장치를 **extended resources**보다 유연하게 요청/할당하는 최신 메커니즘.

Dynamic Resource Allocation. 기존 **extended resources**(예: `nvidia.com/gpu: 1`)보다 풍부한 표현으로, 장치의 속성·공유·파라미터를 지정해 요청/할당하는 최신 메커니즘입니다.

ResourceClaim/ResourceClass 같은 리소스로, 스케줄러가 장치 가용성까지 고려해 배치합니다.

AI/ML·GPU 워크로드에서 주목받는 신기능이에요. (버전에 따라 알파/베타)

참고

nodeName — 스케줄러 우회

nodeName을 직접 박으면 스케줄러를 건너뛰고 그 노드에 강제 배치. 디버깅용/주의 필요.

spec.nodeName에 노드 이름을 직접 적으면, 스케줄러를 **완전히 건너뛰고** kubelet이 그 노드에서 바로 파드를 실행합니다.

```
spec:  
  nodeName: node1
```

주의: 리소스 검사·taint·어피니티를 전부 무시합니다. 그 노드가 없거나 자원이 부족하면 파드가 그냥 실패해요. 디버깅/특수 상황 외엔 권장하지 않습니다.

한눈에 정리

16장 도구 선택 가이드 (최신 권장)

- 전용 노드 격리 → [taint/toleration](#) (+ nodeAffinity) · 특정 노드 유치 → nodeAffinity(또는 간단하면 [nodeSelector](#))
- 파드끼리 모으기 → podAffinity · 균등 분산 →  topologySpreadConstraints ([안티 어피니티](#)보다 우선)
- 자원 경쟁 우선순위 → [PriorityClass](#) · 커스텀 로직 → [스케줄링 프레임워크](#) 플러그인 + 프로파일

이 슬라이드의 용어 풀이

레이블

리소스(노드·파드 등)에 붙이는 key=value 꼬리표예요. 스케줄링 규칙(어피니티·셀렉터)이 이 레이블을 보고 대상을 고릅니다.

스케줄러

"이 파드를 어느 노드에서 실행할까?"만 결정하는 컴포넌트예요. 파드를 직접 만들거나 실행하지는 않고, 조건에 맞는 노드를 골라 nodeName을 채워줘요.

스케줄링 프레임워크

스케줄러 내부를 여러 "확장 지점"(Filter, Score, PostFilter, Bind 등)으로 쪼개고, 각 지점에 플러그인을 끼워 동작을 바꾸는 최신 구조예요. 예전의 predicate/priority를 대체했어요.

안티 어피니티

"같은 종류의 파드가 이미 있는 곳은 피해라"는 규칙이에요. 레플리카를 서로 다른 노드/존에 흩뿌려 한 곳이 죽어도 살아남게 해요.

축출

실행 중인 파드를 노드에서 강제로 내보내는 것(eviction)이에요. NoExecute taint나 우선순위 선점에서 일어나요.

DRA

Dynamic Resource Allocation. GPU 같은 특수 장치를 기존 방식보다 유연하게 요청·할당하는 최신 메커니즘이에요. AI/ML 워크로드에서 주목받아요.

IgnoredDuringExecution

"이미 실행 중이면 무시"라는 뜻이에요. 배치된 뒤 노드 레이블이 바뀌어 조건이 안 맞아도 파드를 쫓아내지 않아요. (실행 중 축출하는 RequiredDuringExecution은 아직 미구현)

nodeName

파드 스펙에 노드 이름을 직접 박아 스케줄러를 건너뛰고 그 노드에 강제 배치하는 필드예요. 리소스·taint·어피니티를 전부 무시해서 위험해요.

nodeSelector

파드에 "이 레이블을 가진 노드에만 가라"고 적는 가장 단순한 방법이에요. 정확히 일치(AND)만 되고, 노드 어피니티가 그 상위호환이에요.

PriorityClass

파드의 중요도(정수 값)를 정의하는 리소스예요. 파드는 priorityClassName으로 이걸 참조하고, 값이 클수록 스케줄링·자원 경쟁에서 우선해요.

taint

영어로 "오염·얼룩"이라는 뜻이에요. 노드에 붙이는 "꺼림칙한 딱지(출입 통제 표식)"라고 보면 돼요. 이 딱지를 "괜찮아"라고 받아주는 toleration이 없는 파드는 그 노드에 배치되지 못해요.

toleration

파드가 노드의 taint를 "괜찮아, 참을 수 있어"라고 받아주는 표식이에요. "견딘다 = 그 노드의 꺼림칙한 조건을 참아낸다". taint와 key/effect가 맞으면 그 노드에 들어갈 자격이 생겨요.

정리 핵심 요약

16장 고급 스케줄링 — 1판 핵심 + 2판/최신 변화 한 장 정리

1판의 3대 도구

- ▶ **테인트 & 톨러레이션** — 노드가 파드를 *밀어냄*.
`key=value:effect(NoSchedule/PreferNoSchedule/NoExecute)`. 전용 노드·특수 HW·장애 축출에 사용
- ▶ **노드 어피니티** — 파드가 노드로 *끌림*. `nodeSelector` 상위호환,
`hard(required)/soft(preferred) + In/NotIn/Exists/Gt/Lt`
- ▶ **파드 (안티)어피니티** — 다른 파드 기준 배치. `topologyKey`로 "같은 곳" 범위(노드/존/리전) 결정. 안티는 흩뿌리기

2판/최신에서 꼭 더해야 할 것 (★)

★

토폴로지 분산 제약

`maxSkew`로 유연한 균등 분산. 안티 어피니티의 분산 용도를 대체.

`maxSkew`로 도메인 간 파드 개수 차이를 제한해 균등하게 분산합니다. `whenUnsatisfiable`로 `hard/soft` 선택. "레플리카를 존마다 고르게"라는 1판 안티 어피니티의 대표 용도를 더 유연하게 대체합니다.

★

스케줄링 프레임워크/프로파일

`predicate/priority` → 플러그인 + 확장 지점 구조.
`schedulerName`으로 프로파일 선택.

`predicate/priority` → 확장 지점
(`Filter/Score/PostFilter/Bind` 등)에 플러그인을 끼우는 구조. 한 스케줄러가 여러 프로파일을 운용하고, 파드는 `schedulerName`으로 선택합니다.

★

우선순위 & 선점

`PriorityClass`로 중요도 부여, 자리 없으면 낮은 파드 축출.

파드에 중요도(정수)를 부여하고, 높은 파드가 자리 없으면 낮은 파드를 축출(선점). `preemptionPolicy: Never`로 선점 없이 우선 배치만 받을 수도 있습니다.

★

레이블 변경 & 신규 도구

`topology.kubernetes.io/*` 표준화, `control-plane taint`, `Descheduler`·`DRA`.

- `failure-domain.beta.kubernetes.io/zone` → `topology.kubernetes.io/zone`
- `beta.kubernetes.io/arch|os` → `kubernetes.io/arch|os`
- 마스터 taint `master` → `control-plane`
- 신규 도구: `Descheduler`(재분산), `DRA`(장치 할당)

🎯 **한 줄로:** 16장은 "스케줄러에게 내 의도를 주입하는 법"이에요. 밀어내기(**taint**) · 끌어당기기(**node/pod affinity**) · 흩뿌리기(**anti-affinity/분산 제약**) · 우선순위(**preemption**) 네 방향을 조합하면, 가용성·성능·비용·격을 원하는 대로 설계할 수 있습니다.

👏 **발표 마무리:** 1판으로 개념을 익히되, 실습은 **topology.kubernetes.io 레이블**과 **topologySpreadConstraints**로 — 그게 1판과 최신의 가장 큰 실무 차이입니다. 감사합니다!

이 슬라이드의 용어 풀이

노드 어피니티

파드가 "이런 레이블을 가진 노드에 가고 싶다"를 표현하는 규칙이에요. nodeSelector보다 표현력이 좋아 In/NotIn 등 연산자와 필수/선택 두 단계를 지원해요.

레이블

리소스(노드·파드 등)에 붙이는 key=value 꼬리표예요. 스케줄링 규칙(어피니티·셀렉터)이 이 레이블을 보고 대상을 고릅니다.

선점

높은 우선순위 파드가 배치될 자리가 없을 때, 스케줄러가 낮은 우선순위 파드를 축출해 자리를 만드는 것이예요(preemption).

스케줄러

"이 파드를 어느 노드에서 실행할까?"만 결정하는 컴포넌트예요. 파드를 직접 만들거나 실행하지는 않고, 조건에 맞는 노드를 골라 nodeName을 채워줘요.

스케줄링 프레임워크

스케줄러 내부를 여러 "확장 지점"(Filter, Score, PostFilter, Bind 등)으로 쪼개고, 각 지점에 플러그인을 끼워 동작을 바꾸는 최신 구조예요. 예전의 predicate/priority를 대체했어요.

안티 어피니티

"같은 종류의 파드가 이미 있는 곳은 피해라"는 규칙이에요. 레플리카를 서로 다른 노드/존에 흩뿌려 한 곳이 죽어도 살아남게 해요.

축출

실행 중인 파드를 노드에서 강제로 내보내는 것(eviction)이에요. NoExecute taint나 우선순위 선점에서 일어나요.

테인트

노드에 붙이는 "출입 통제 표식"이에요(taint). key=value:effect 형태로, 이 표식을 건디는 toleration이 없는 파드는 그 노드에 못 들어와요. 어피니티와 반대로 "밀어내는" 방향이에요.

토폴로지 분산 제약

파드를 존/노드에 고르게 퍼 바르는 최신 도구예요(topologySpreadConstraints). maxSkew로 "도메인 간 개수 차이"를 제한해, 안티 어피니티보다 유연하게 균등 분산해요.

톨러레이션

파드가 노드의 taint(꺼림칙한 딱지)를 "괜찮아, 참고 들어갈게"라고 받아주는 표식이에요(toleration). "견딘다"는 = "그 딱지를 참아낸다"는 뜻. taint와 짝이 맞아야 그 노드에 들어갈 "자격"이 생겨요. 단, 꼭 그 노드로 간다는 뜻은 아니예요.

DRA

Dynamic Resource Allocation. GPU 같은 특수 장치를 기존 방식보다 유연하게 요청·할당하는 최신 메커니즘이에요. AI/ML 워크로드에서 주목받아요.

effect

taint가 파드에 미치는 효과예요. NoSchedule(새 배치 금지) / PreferNoSchedule(되도록 금지) / NoExecute(실행 중인 것도 축출) 세 가지가 있어요.

maxSkew

토폴로지 분산 제약에서 "가장 많은 도메인과 가장 적은 도메인의 파드 개수 차이" 허용 최대치예요. 작을수록 더 뽁뽁하게 균등해져요.

nodeSelector

파드에 "이 레이블을 가진 노드에만 가라"고 적는 가장 단순한 방법이에요. 정확히 일치(AND)만 되고, 노드 어피니티가 그 상위호환이에요.

NoExecute

taint 효과 중 하나. 새 파드 배치를 막는 건 물론, "이미 실행 중인데 견디지 못하는 파드까지 쫓아내요(축출)". 노드 장애 시 자동 축출에 쓰여요.

NoSchedule

taint 효과 중 하나. 견디지 못하는 파드를 그 노드에 "새로 배치하지 않음"이에요. 이미 떠 있는 파드는 건드리지 않아요.

preemptionPolicy

우선순위 파드가 "남을 쫓아낼지" 정하는 정책이에요. PreemptLowerPriority(쫓아냄, 기본) / Never(우선 배치만 받고 선점은 안 함).

PreferNoSchedule

taint 효과 중 하나. "되도록 배치하지 말되, 다른 자리가 없으면 허용"하는 약한(soft) 버전이에요.

PriorityClass

파드의 중요도(정수 값)를 정의하는 리소스예요. 파드는 priorityClassName으로 이걸 참조하고, 값이 클수록 스케줄링·자원 경쟁에서 우선해요.

taint

영어로 "오염·얼룩"이라는 뜻이에요. 노드에 붙이는 "꺼림칙한 딱지(출입 통제 표식)"라고 보면 돼요. 이 딱지를 "괜찮아"라고 받아주는 toleration이 없는 파드는 그 노드에 배치되지 못해요.

topologyKey

파드 어피니티에서 "같은 곳"의 범위를 정하는 레이블 키예요. hostname이면 "같은 노드", zone이면 "같은 가용 영역"처럼 도메인 크기를 바꿔요.

whenUnsatisfiable

분산 제약을 못 맞출 때 어떻게 할지예요. DoNotSchedule(못 맞추면 Pending, hard) / ScheduleAnyway(그래도 배치, soft) 둘 중 하나예요.